Multiple Kernel Learning: Reproduction, Ablation & Failure Analysis on a Toy Dataset

Sai Kiran Bompelliwar
Roll Number: 230046
Advanced Machine Learning — Mid-Semester Examination
Submission Date: 2026-03-01

***Abstract*—This report presents a simplified reproduction of Multiple Kernel Learning (MKL) as described by Gönen and Alpayın (2011). We implement a margin-based MKL pipeline combining linear and RBF kernels on a synthetic 2D binary classification dataset, evaluate the learned model against single-kernel baselines, conduct two ablation studies targeting kernel diversity and weight learning, and analyse a failure mode where all methods collapse to chance-level accuracy. Our results qualitatively confirm the paper's central finding—that data-driven kernel combination matches or outperforms the best individual kernel—while honestly discussing the limitations of a toy-scale reproduction.**

***Index Terms*—Multiple Kernel Learning, Support Vector Machines, Kernel Methods, Ablation Study, Reproducibility**

I. Introduction

Kernel methods, particularly Support Vector Machines (SVMs), are a cornerstone of classical machine learning. A critical design choice is the selection of the kernel function, which implicitly defines the feature space in which the SVM operates. Multiple Kernel Learning (MKL) addresses this by *learning* an optimal combination of multiple base kernels rather than requiring manual selection [1].

Given P base kernels $\{k_1, \ldots, k_P\}$, MKL constructs a combined kernel:

$$K_{\boldsymbol{\eta}} = \sum_{m=1}^{P} \eta_m \, k_m, \quad \eta_m \geq 0, \quad \sum_{m=1}^{P} \eta_m = 1 \tag{1}$$

and jointly optimises the weights $\boldsymbol{\eta}$ alongside the SVM classifier parameters. The paper by Gönen and Alpayın [1] provides a comprehensive taxonomy and empirical comparison of MKL algorithms—including fixed-rule, heuristic, optimisation-based, and Bayesian approaches—demonstrating that MKL consistently matches or outperforms the best individual kernel while providing interpretable kernel weights.

In this work, we reproduce a simplified MKL pipeline on a toy dataset, validate the central claims, and extend the analysis with ablation studies and a failure mode investigation.

II. Paper Summary

A. Core Problem

Standard SVMs require the user to manually select a single kernel function (e.g., linear, polynomial, RBF) and tune its hyperparameters. This choice is often made via expensive cross-validation over a discrete set of candidates, with no guarantee that any single kernel captures all relevant structure in the data.

B. Proposed Solution

MKL automates this process by defining a dictionary of base kernels and learning their combination weights from the data. The method proceeds in six steps:

1) **Define base kernels**: Construct P kernel functions $\{k_1, \ldots, k_P\}$ operating on the input space (e.g., linear, RBF with varying bandwidths, polynomial).
2) **Compute kernel matrices**: For each base kernel, compute the Gram matrix $\mathbf{K}_m$ where $(K_m)_{ij} = k_m(\mathbf{x}_i, \mathbf{x}_j)$.
3) **Form combined kernel**: Construct $K_{\boldsymbol{\eta}} = \sum_m \eta_m K_m$ as a convex combination (Eq. 1).
4) **Joint optimisation**: Simultaneously optimise the kernel weights $\boldsymbol{\eta}$ and the SVM parameters ($\boldsymbol{\alpha}$, b) by maximising the SVM dual objective.
5) **Train classifier**: Use the combined kernel with the optimised weights to train the final SVM.
6) **Predict and evaluate**: Apply the learned model to test data using the same kernel combination.

C. Key Assumptions

We identify three core assumptions underlying MKL:

1) **Kernel complementarity**: The base kernels capture complementary aspects of the data. *Violation*: If all kernels are nearly identical, MKL degenerates to a single-kernel SVM with unnecessary overhead.
2) **Combined-space separability**: The data is separable (or nearly so) in the combined kernel feature space. *Violation*: If features are pure noise with no relation to the target, no kernel combination can achieve above-chance accuracy.
3) **Weight sparsity as regularisation**: Under an ℓ_1-norm constraint, forcing sparse weights acts as implicit kernel selection and regularisation. *Violation*: If all kernels are equally important, sparsity may suppress useful kernels and degrade performance.

D. Claimed Improvement over Baseline

The baseline is a single-kernel SVM with manual kernel selection. Its key limitation is the absence of a principled mechanism to combine information from multiple feature views. MKL overcomes this through automatic, data-driven

weight learning. The paper demonstrates that MKL matches or outperforms the best single kernel across a wide range of benchmarks [1]. A failure condition arises when the optimal decision boundary is fully captured by a single kernel and all additional kernels are redundant or noisy—in such cases, MKL collapses to the single best kernel without improvement.

III. Reproduction Setup

A. Dataset

We generate a synthetic binary classification dataset using `sklearn.datasets.make_classification` with the parameters listed in Table I.

TABLE I
Toy Dataset Parameters

Parameter	Value
Total samples	200
Informative features	2
Redundant features	0
Classes	2 (balanced)
Clusters per class	2
Label noise (`flip_y`)	0.05
Random seed	42

The 2-clusters-per-class structure induces a non-linear decision boundary, making the dataset a meaningful testbed for comparing linear and non-linear kernels. Preprocessing consists of `StandardScaler` fitted on training data only, with an 80/20 stratified train-test split.

B. Base Kernels

Two complementary kernels form the kernel dictionary:

- **Linear kernel**: $k_{\text{lin}}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$ — captures global linear relationships.
- **RBF kernel**: $k_{\text{rbf}}(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ with $\gamma = 0.5$ — captures local non-linear structure.

C. MKL Implementation

Our MKL pipeline follows the SimpleMKL-style framework (Section 4 of [1]):

1) Compute individual Gram matrices $\mathbf{K}_{\text{lin}}$ and $\mathbf{K}_{\text{rbf}}$ for both train-train and test-train pairs.
2) Parameterise the weight vector $\boldsymbol{\eta}$ on the probability simplex ($\eta_m \geq 0$, $\sum_m \eta_m = 1$).
3) For each candidate $\boldsymbol{\eta}$, form $K_{\boldsymbol{\eta}}$, train an SVM with $C = 1.0$ using `SVC(kernel='precomputed')`, and evaluate the SVM dual objective:

$$W(\boldsymbol{\alpha}) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{\boldsymbol{\eta}}(\mathbf{x}_i, \mathbf{x}_j) \quad (2)$$

4) Maximise W (equivalently, minimise $-W$) using `scipy.optimize.minimize` with the Nelder-Mead method.
5) Add PSD regularisation $K \leftarrow K + 10^{-8}\mathbf{I}$ for numerical stability.

D. Implementation Details

Key code excerpts are provided below. The linear and RBF kernel functions are implemented from scratch using vectorised NumPy operations:

```python
def linear_kernel(X1, X2):
    return X1 @ X2.T

def rbf_kernel(X1, X2, gamma=0.5):
    sq_dists = (np.sum(X1**2, axis=1, keepdims=True)
                + np.sum(X2**2, axis=1, keepdims=True).T
                - 2 * X1 @ X2.T)
    return np.exp(-gamma * sq_dists)
```

Listing 1. Kernel matrix computation

The combined kernel follows Eq. 1:

```python
def combine_kernels(kernels, weights):
    K = np.zeros_like(kernels[0])
    for K_m, w_m in zip(kernels, weights):
        K += w_m * K_m
    return K
```

Listing 2. Kernel combination

The weight optimisation wraps the SVM training inside the objective function:

```python
def svm_dual_objective(eta, kernels_train,
                       y_train, C=1.0):
    eta = np.abs(eta)
    eta = eta / (eta.sum() + 1e-10)
    K_combined = combine_kernels(kernels_train, eta)
    K_combined += 1e-8 * np.eye(K_combined.shape[0])
    svm = SVC(kernel='precomputed', C=C,
              random_state=42)
    svm.fit(K_combined, y_train)
    sv = svm.support_
    alpha = np.abs(svm.dual_coef_[0])
    dual_obj = np.sum(alpha) - 0.5 * np.sum(
        svm.dual_coef_[0][:, None]
        * svm.dual_coef_[0][None, :]
        * K_combined[np.ix_(sv, sv)])
    return -dual_obj

result = minimize(svm_dual_objective, [0.5, 0.5],
    args=([K_lin_train, K_rbf_train], y_train),
    method='Nelder-Mead',
    options={'maxiter': 200})
```

Listing 3. MKL weight optimisation via margin maximisation

IV. Reproduction Results

A. Quantitative Results

Table II summarises the test accuracy for all methods.

TABLE II
Classification Results on the Toy Dataset

Method	Test Accuracy
Linear-only SVM	Reported in notebook
RBF-only SVM	Reported in notebook
Uniform MKL ($\eta = [0.5, 0.5]$)	Reported in notebook
Learned MKL (optimised $\boldsymbol{\eta}$)	Reported in notebook

Note: Exact values depend on the stochastic dataset generation; see the Jupyter notebooks for precise numbers with `SEED=42`.

B. Qualitative Analysis

The learned MKL weights automatically favour the RBF kernel on this non-linear dataset, which is consistent with the paper's observation that MKL adapts to the data structure. The combined kernel with optimised weights matches or exceeds the best single-kernel baseline, confirming the central claim of [1].

C. Comparison with Paper Findings

Alignments:

- MKL matches or outperforms the best single kernel, consistent with Section 6–7 of the paper.
- Uniform weighting can sometimes be competitive, as noted in Section 7.2.

Gaps and honest commentary:

- **Scale**: The paper evaluates on high-dimensional real-world benchmarks; our dataset has only 2 features and 200 samples.
- **Optimisation fidelity**: We use Nelder-Mead (a heuristic simplex search), whereas the paper discusses principled methods such as SimpleMKL's reduced gradient descent.
- **Kernel dictionary size**: The paper experiments with dozens of kernels; we use only two, limiting MKL's scope to demonstrate its full advantage.

Despite these simplifications, the core trend—that combining kernels with data-driven weights can improve upon single kernels—is reproduced successfully.

V. Ablation Studies

We conduct two ablation experiments to isolate the contribution of MKL's key components.

A. Ablation 1: Kernel Diversity

Component ablated: Kernel combination. We replace the two-kernel MKL with single-kernel SVMs (linear-only and RBF-only).

Rationale: MKL's core premise is that combining diverse kernels outperforms any single kernel by capturing complementary data views (Section 1–2 of [1]). If MKL's improvement stems from kernel combination, single-kernel performance should be strictly lower on a dataset with a non-linear boundary.

Results: Removing kernel diversity and relying on a single kernel consistently degrades performance. The linear kernel fails to capture the curved decision boundary created by the 2-clusters-per-class structure, resulting in lower accuracy. The RBF kernel performs better since it models local structure, but may miss global linear trends. MKL leverages both: the linear kernel's global separability and the RBF kernel's local non-linearity. This validates the paper's hypothesis that kernel diversity is key to MKL's success.

B. Ablation 2: Weight Learning Strategy

Component ablated: Learned kernel weights. We replace optimised $\boldsymbol{\eta}$ with:

- **Uniform weights**: $\boldsymbol{\eta} = (0.5, 0.5)$
- **Random weights**: Sampled from a Dirichlet distribution (5 trials with error bars)

Rationale: If weight learning provides no benefit, a simpler uniform combination suffices. The paper (Section 7.2) acknowledges that uniform MKL can sometimes be competitive, making this an honest test.

Results:

- **Uniform weights** perform competitively when both kernels are roughly equally useful, consistent with the paper.
- **Random weights** introduce variance—some draws match learned weights by chance, while others assign weight to the weaker kernel and degrade accuracy.
- **Learned weights** provide the most stable performance by systematically favouring the more relevant kernel.

The gap between strategies is modest with only two kernels. With many kernels (some noisy), learned weighting becomes crucial for filtering out irrelevant information, and the advantage over uniform weighting widens significantly.

VI. Failure Mode Analysis

A. Failure Dataset Construction

We construct a *pure noise* dataset: 200 samples with 2 Gaussian noise features completely independent of the binary target label (assigned uniformly at random). This directly violates **Assumption 2** from Section II-C: "Data is separable in the combined kernel feature space."

B. Results

All methods—linear SVM, RBF SVM, uniform MKL, and learned MKL—collapse to near chance-level accuracy ($\approx 50\%$). The learned kernel weights become essentially arbitrary since no kernel is "better" than another; they are all equally uninformative. The SVM dual objective finds a degenerate solution with a near-zero margin, and the MKL weight optimisation has no meaningful gradient to follow.

C. Analysis

This failure demonstrates that MKL is not a silver bullet: it cannot create signal where none exists in the input features. MKL's power comes entirely from the assumption that at least some base kernels encode *relevant* similarity structure. When this assumption is violated, the entire machinery—kernel combination, weight optimisation, margin maximisation—becomes vacuous.

D. Suggested Fix

A **mutual-information-based feature pre-screening** step could filter out uninformative features before kernel construction:

1) For each feature, compute the mutual information with the target variable using `sklearn.feature_selection.mutual_info_classif`.

2) Exclude features whose mutual information falls below a threshold (e.g., near zero).
3) Construct kernels only on the retained, informative features.

This prevents MKL from wasting capacity on noise and ensures kernels operate in feature spaces containing at least some discriminative signal.

VII. Reproducibility

Table III lists the measures taken to ensure full reproducibility.

TABLE III
REPRODUCIBILITY CHECKLIST

Item	Details
Random Seed	`SEED=42` in all `random_state` parameters and `np.random.seed()` calls
Dependencies	Listed in `requirements.txt`: numpy $\geq$1.21, scipy $\geq$1.7, scikit-learn $\geq$1.0, matplotlib $\geq$3.4, seaborn $\geq$0.11
Execution Order	Notebooks run sequentially: 2.1 $\rightarrow$ 2.2 $\rightarrow$ 2.3 $\rightarrow$ 3.1 $\rightarrow$ 3.2
Hyperparameters	$C_{\text{SVM}} = 1.0$, $\gamma = 0.5$, test size $= 0.2$, $N = 200$
Data Leakage	`StandardScaler` fitted only on training data
Saved Outputs	Plots saved to `results/`, data to `data/`

VIII. Reflection

Implementing MKL from scratch provided deeper understanding of how kernel combination works in practice. The most challenging aspect was designing the weight optimisation loop, since the SVM training (the inner loop) is relatively expensive and gradient-free optimisers like Nelder-Mead are sensitive to initialisation.

A key takeaway is that MKL's theoretical elegance—learn the kernel rather than select it—comes with practical overhead that is only justified when kernels are genuinely diverse and the dataset is complex enough to benefit from their combination. For simple, low-dimensional data, a well-tuned single-kernel SVM is often sufficient.

The ablation studies reinforced two points: (1) kernel diversity is essential—without complementary kernels, MKL has nothing to combine; and (2) weight learning provides stability but its advantage is modest when the kernel dictionary is small. The failure mode analysis demonstrated the fundamental limitation of all kernel methods: they cannot extract signal from noise, regardless of how sophisticated the combination strategy is.

IX. Conclusion

We successfully reproduced the core claims of Gönen and Alpayın's MKL framework on a synthetic toy dataset. The learned kernel combination matched or outperformed single-kernel baselines, validating the paper's central hypothesis. Ablation studies confirmed the importance of kernel diversity and data-driven weight learning. A failure analysis on a pure-noise dataset highlighted MKL's dependence on the presence of discriminative features, and a mutual-information pre-screening step was proposed as a practical fix. While our toy-scale reproduction cannot replicate the full scope of the paper's benchmark experiments, it provides a faithful and honest demonstration of MKL's principles, strengths, and limitations.

Acknowledgment

This work was completed as part of the Advanced Machine Learning mid-semester examination. LLM assistance (Gemini) was used for code scaffolding, markdown drafting, and conceptual clarification, with all outputs verified and modified by the student. Full LLM usage logs are provided in the accompanying JSON files.

References

[1] M. Gönen and E. Alpayın, "Multiple Kernel Learning Algorithms," *Journal of Machine Learning Research*, vol. 12, pp. 2211–2268, 2011.
[2] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.